

Weekly Benchmark Summary: Local Emotion Analysis Prototype

Atabey Tarik Akkum

1 July 2026

Purpose of this week

The main goal this week was to move the prototype from mock inference to real on-device model comparison for facial emotion recognition. The intended direction was to test a strong task-specific emotion model, compare it against a local multimodal model, and make benchmark outputs exportable for later analysis and supervisor discussion.

The initial plan was to use an ONNX version of the official ResEmoteNet model, because the paper reports strong AffectNet performance. The ResEmoteNet paper reports 72.93% accuracy on AffectNet and links to a public GitHub repository. During implementation, that linked repository was not available from our environment, so the original implementation/checkpoint could not be used directly. As a practical replacement, I used a Kaggle notebook/checkpoint that provides a ResEmoteNet implementation trained on FER2013, RAF-DB, and AffectNet, and converted that model to ONNX.

I also noted POSTER v2 as another relevant high-performing facial expression recognition project. It is worth keeping as a possible future comparison point, but I did not integrate it this week because the current priority was to get a clean Android ONNX/Gemma benchmark loop working first.

Implemented model paths

- **EmotiEff / HSEmotion path:** Integrated ONNX Runtime inference for the EmotiEff-style EfficientNet models. The strongest currently tested baseline is `enet_b2_7`, which is a 7-class emotion classifier and therefore directly matches the benchmark label space.
- **ResEmoteNet path:** The first ResEmoteNet checkpoint I found behaved like a 64x64 input reproduction and gave clearly weaker results. The Kaggle checkpoint was more suitable because it follows a 224x224 RGB preprocessing path, which is closer to the AffectNet setup described in the ResEmoteNet paper. This model was converted to `ResEmoteNetKaggle_224x224.onnx` and integrated as a separate variant.
- **ResEmoteNet validation:** I also benchmarked the earlier 64x64 BS32 variant to make the difference explicit. It is included below as `ResEmoteNet BS32 64x64` and is substantially weaker than the Kaggle 224x224 checkpoint.
- **Gemma 4 E2B path:** Integrated Gemma 4 E2B locally through LiteRT-LM. This required storing the 2.4 GB `.litemlm` model inside the app-private storage, because shared storage paths such as `/sdcard/Demo` are not readable by the native runtime on GrapheneOS. Single-image inference and full benchmark execution now work.
- **Export path:** Benchmark results are written as CSV and JSON. The app now also has a share/export button so results can be moved out of app-private storage without ADB.

Gemma task configuration

Gemma is not hardcoded only as an emotion model. The current emotion task is configured through `assets/gemma_tasks/emotion_classification.json`. For benchmarking, the prompt was intentionally shortened to reduce token overhead and runtime failures. The current task asks Gemma to return label-only JSON:

```
{"label": "happy"}
```

The Dart parser then converts this into the same seven-label output schema used by EmotiEff and ResEmoteNet. This keeps benchmark exports structurally comparable, although Gemma currently produces one-hot confidence scores rather than calibrated class probabilities.

Benchmark setup

All models were benchmarked on the same balanced AffectNet-7 subset:

- Dataset: `affectnet7_25.zip`
- Classes: angry, disgust, fear, happy, neutral, sad, surprise
- Examples: 25 images per class, 175 images total
- Excluded class: contempt
- All runs completed with 0 failed examples

This subset is useful for fast iteration and debugging, but it is not large enough for final claims. It should be treated as an implementation benchmark and early indication, not as the final evaluation.

Benchmark results

Model	Accuracy	Macro-F1	Avg. latency	Median latency	Failures
EmotiEff EfficientNet B2 7-class	68.57%	68.54%	1219 ms	590 ms	0
ResEmoteNet Kaggle 224	64.57%	64.36%	1339 ms	713 ms	0
ResEmoteNet BS32 64x64	47.43%	47.38%	1182 ms	534 ms	0
Gemma 4 E2B	42.86%	35.18%	6931 ms	6422 ms	0

Per-class F1

Class	EmotiEff B2 7	ResEmoteNet 224	ResEmoteNet 64	Gemma 4 E2B
angry	60.71%	61.90%	41.03%	53.06%
disgust	66.67%	71.11%	43.24%	7.41%
fear	75.86%	51.16%	60.87%	0.00%
happy	82.61%	82.14%	56.41%	75.00%
neutral	55.81%	57.53%	31.82%	45.28%
sad	76.60%	66.67%	46.32%	20.69%
surprise	61.54%	60.00%	52.00%	44.83%

Interpretation

EmotiEff EfficientNet B2 7-class is currently the strongest and most practical local baseline. It reached 68.57% accuracy and 68.54% macro-F1 on the 175-image AffectNet-7 subset. Its average latency was about 1.22 seconds per image, with a much lower median latency of 590 ms. This makes it the best current candidate for reliable on-device facial emotion recognition.

ResEmoteNet Kaggle 224 is competitive but weaker than EmotiEff in this implementation. It reached 64.57% accuracy and 64.36% macro-F1. This is much better than the 64x64 BS32 checkpoint, which reached only 47.43% accuracy and 47.38% macro-F1 on the same benchmark. The comparison supports the conclusion that preprocessing and checkpoint provenance matter substantially for ResEmoteNet.

For the Kaggle ResEmoteNet checkpoint, I also checked the PyTorch-side behavior against the ONNX/mobile behavior on the same dataset. The result was effectively the same after conversion, so the phone/ONNX path does not appear to reduce accuracy. The main cost of running the converted model on the phone is latency, not classification quality.

Gemma 4 E2B works locally and can process the same benchmark images through a configurable multimodal prompt. However, it is much slower and less accurate for this narrow task. It reached 42.86% accuracy and 35.18% macro-F1, with an average latency of about 6.93 seconds per image. This result is still useful scientifically: it shows that a general multimodal model is easy to adapt to a new trait through prompting, but for narrow facial emotion classification it currently underperforms specialized FER models.

Technical lessons

- Specialized emotion models remain much better for this specific trait. EmotiEff is both faster and more accurate than Gemma 4 E2B on this benchmark.
- The 224x224 ResEmoteNet checkpoint was much more plausible than the earlier 64x64 reproduction. The 64x64 result is useful as a negative control.
- Gemma 4 E2B is valuable as an extensible analysis path. It can be controlled through task JSON and can later be reused for screenshots, PDFs, image context, and other privacy-relevant traits.
- Gemma model storage matters on Android. The `.litertlm` file has to be in app-private storage for the native runtime. Shared storage caused permission problems on GrapheneOS.
- Gemma benchmarking needs stateless requests. Reusing the same conversation for many images caused runtime failures; the current implementation keeps the engine loaded but resets the conversation for each image.
- Exportability is now solved. CSV and JSON exports can be shared from the app and are also available for offline analysis.

Current recommendation

For the next project step, I would keep EmotiEff B2 7 as the main specialized emotion baseline, keep ResEmoteNet Kaggle 224 as a secondary comparison point, keep the 64x64 ResEmoteNet result as evidence for why that checkpoint should not be used as the main baseline, and keep Gemma 4 E2B as the general multimodal local inference path. The current results support the thesis direction: task-specific models are better for a known trait, while Gemma is more interesting for flexible and broader local privacy inference.

Sources and references

- ResEmoteNet paper: <https://arxiv.org/html/2409.10545v1>
- ResEmoteNet linked repository from the paper: <https://github.com/ArnabKumarRoy02/ResEmoteNet>
- Kaggle ResEmoteNet notebook used for the practical checkpoint: <https://www.kaggle.com/code/tngthnhvui/resemodenet-rafdb-fer-affectnet-25/notebook>
- POSTER v2 repository noted for possible later comparison: https://github.com/talented-q/poster_v2
- EmotiEffLib / HSEmotion repository: <https://github.com/sb-ai-lab/EmotiEffLib>
- Google AI Edge Gallery / Gemma local runtime reference: <https://github.com/google-ai-edge/gallery>
- Current task configuration: `assets/gemma_tasks/emotion_classification.json`
- Exported benchmark files: `exports/Gemma4`, `exports/enet_b2_7`, `exports/resemodenet_kaggle_224`, `exports/resemodenet_bs32_64`